

中山大學
SUN YAT-SEN UNIVERSITY

并行程序设计与算法实验 |Lab0

环境设置与串行矩阵乘法

学　　号: 23336179

姓　　名: 马福泉

专　　业: 计算机科学与技术

指 导 教 师: 吴迪刘学正

起 始 日 期: 2026 年 3 月 26 日

结 束 日 期: 2026 年 3 月 27 日

实 验 地 点: B202

目录

1 实验目的 **2**

2 实验内容 **2**

3 实验过程 **2**

3.1 实验环境 2
3.2 矩阵乘法定义与性能指标 3
3.3 各版本实现说明 3
3.3.1 版本 1：Python 基准版本 3
3.3.2 版本 2：C 语言基础版本 (i-j-k) 3
3.3.3 版本 3：C 语言循环重排版本 (i-k-j) 4
3.3.4 版本 4：C 语言编译优化版本 4
3.3.5 版本 5：C 语言循环展开版本 4
3.3.6 版本 6：Intel MKL 库版本 4

4 实验结果 **5**

4.1 测试参数 5
4.2 性能对比表格 5
4.3 运行结果截图 5

5 实验分析 **5**

5.1 循环顺序对性能的影响 5
5.2 编译优化的影响 6
5.3 循环展开的影响 6
5.4 MKL 库的性能优势 6
5.5 Python NumPy 的性能表现 6
5.6 性能优化总结 7

6 实验总结 **7**

6.1 注意事项 7
6.2 心得体会 7
6.3 参考资料 7

1 实验目的

- 理解并行程序设计的基本概念与理论。
- 掌握使用并行编程模型实现常见算法的能力。
- 学习评估并行程序性能的指标及其优化方法。

2 实验内容

- 设计并实现以下矩阵乘法版本：
  - 使用 C 语言实现一个串行矩阵乘法。
  - 比较不同编译选项、实现方式、算法或库对性能的影响：
    * 使用 Python 实现的矩阵乘法。
    * 使用 C 实现的基本矩阵乘法。
    * 调整循环顺序优化矩阵乘法。
    * 应用编译优化提高性能。
    * 使用循环展开技术优化矩阵乘法。
    * 使用 Intel MKL 库进行矩阵乘法运算。
- 生成随机矩阵 A 和 B，进行矩阵乘法运算得到矩阵 C。
- 衡量各版本的运行时间、加速比、浮点性能等。
- 分析不同实现版本对性能的影响。

3 实验过程

3.1 实验环境

本实验在 Windows Subsystem for Linux (WSL) - Ubuntu 22.04 环境下进行，使用 GCC 编译器和 Intel MKL 数学库。

CPU 信息：AMD Ryzen 9 7940HS（Zen 4 架构），物理核心数 8，支持 FMA 和 AVX-512 指令集。

理论峰值性能计算：

峰值 GFLOPS = 5.2 GHz × 8 核心 × 4 (双 FMA 端口) × 8 (AVX-512 双精度) = 1331.2 GFLOPS

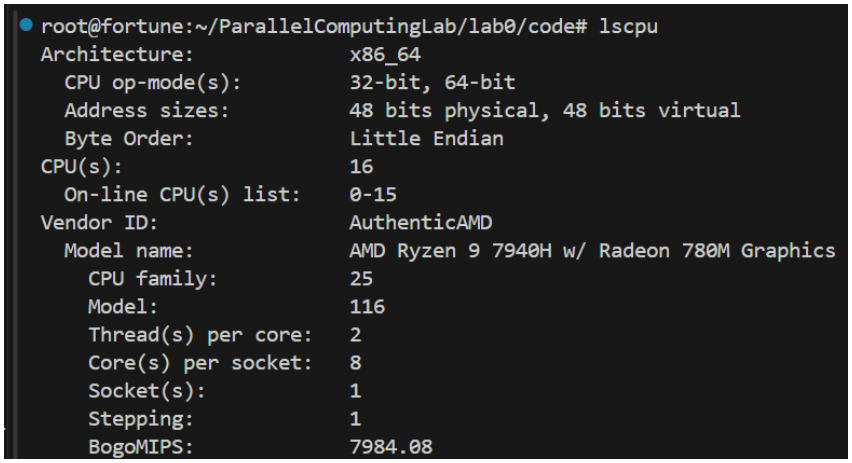


图 1: CPU 信息截图 (lscpu 输出)

3.2 矩阵乘法定义与性能指标

对于矩阵 A $(M \times K)$ 和矩阵 B $(K \times N)$，其乘积 $C = A \times B$ 为 $M \times N$ 矩阵：

$$C[i][j] = \sum_{k=0}^{K-1} A[i][k] \times B[k][j]$$

对于 $N \times N$ 方阵乘法，浮点运算次数为 $2N^3$。性能指标计算如下：

- **GFLOPS**：$\text{GFLOPS} = \dfrac{2 \times N^3}{\text{运行时间} \times 10^9}$
- **相对加速比**：前一版本运行时间 / 当前版本运行时间
- **绝对加速比**：版本 1 运行时间 / 当前版本运行时间
- **峰值性能百分比**：当前 GFLOPS / CPU 峰值 GFLOPS $\times 100\%$

3.3 各版本实现说明

3.3.1 版本 1：Python 基准版本

使用 NumPy 库的 `np.dot()` 函数进行矩阵乘法。NumPy 底层调用了优化的 BLAS 库，作为性能基准参考。

Listing 1: Python 基准版本核心代码

```
import numpy as np
C = np.dot(A, B)
```

3.3.2 版本 2：C 语言基础版本 (i-j-k)

最基础的 C 语言实现，使用 i-j-k 三重循环顺序。

Listing 2: i-j-k 循环顺序

```
for (int i = 0; i < N; i++)
    for (int j = 0; j < N; j++)
```

```
        for (int k = 0; k < N; k++)
            C[i][j] += A[i][k] * B[k][j];
```

3.3.3 版本 3：C 语言循环重排版本 (i-k-j)

改变循环顺序为 i-k-j，优化 Cache 命中率。

Listing 3: i-k-j 循环顺序

```
for (int i = 0; i < N; i++)
    for (int k = 0; k < N; k++)
        for (int j = 0; j < N; j++)
            C[i][j] += A[i][k] * B[k][j];
```

优化原理：

- i-j-k 顺序中，`B[k][j]` 是列优先访问，Cache 命中率低
- i-k-j 顺序中，`C[i][j]` 和 `B[k][j]` 都是行优先访问，Cache 命中率高

3.3.4 版本 4：C 语言编译优化版本

在版本 3 基础上使用 GCC `-O3` 编译选项。`-O3` 优化包含：循环展开、函数内联、自动向量化、寄存器优化等。

3.3.5 版本 5：C 语言循环展开版本

在版本 4 基础上手动进行循环展开（Loop Unrolling）。

Listing 4: 循环展开示例

```
#define UNROLL 4
for (int j = 0; j <= N - UNROLL; j += UNROLL) {
    C[i][j]     += a_ik * B[k][j];
    C[i][j + 1] += a_ik * B[k][j + 1];
    C[i][j + 2] += a_ik * B[k][j + 2];
    C[i][j + 3] += a_ik * B[k][j + 3];
}
```

优化原理：减少循环控制开销，增加指令级并行性。

3.3.6 版本 6：Intel MKL 库版本

使用 Intel MKL 库的 `cblas_dgemm` 函数。

Listing 5: MKL cblas_dgemm 调用

```
#include <mkl.h>
cblas_dgemm(CblasRowMajor, CblasNoTrans, CblasNoTrans,
            N, N, N, 1.0, A, N, B, N, 0.0, C, N);
```

MKL 优化技术：针对 Intel CPU 优化的 BLAS 例程、自动向量化、多线程并行、Cache 优化。

4　实验结果

4.1　测试参数

- 矩阵大小：$N = 1024$
- 浮点运算次数：$2 \times N^3 = 2,147,483,648$

4.2　性能对比表格

表 1: 矩阵乘法性能对比

版本	实现描述	运行时间 (s)	相对加速比	绝对加速比	GFLOPS	峰值百分比
1	Python (NumPy)	0.7295	1.00	1.00	2.94	0.22%
2	C (i-j-k)	7.2863	0.10	0.10	0.29	0.02%
3	C (i-k-j)	4.8471	1.50	0.15	0.44	0.03%
4	C (-O3)	0.4860	9.98	1.50	4.42	0.33%
5	C (循环展开)	0.3401	1.43	2.15	6.31	0.47%
6	Intel MKL	0.1017	3.34	7.17	21.11	1.59%

4.3　运行结果截图

```
------------------------------------------------
V1: Python 基准版本
------------------------------------------------
矩阵大小: N = 1024
浮点运算次数: 2 * 1024^3 = 2,147,483,648
运行时间: 0.729513 秒
GFLOPS: 2.9437

------------------------------------------------
运行: V2: C 语言基础版本 (i-j-k)
------------------------------------------------
矩阵大小: N = 1024
浮点运算次数: 2 * 1024^3 = 2147483648

========== V2: C 语言基础版本 (i-j-k) 结果 ==========
矩阵大小: 1024 x 1024
运行时间: 7.286258 秒
GFLOPS: 0.2947
================================================

------------------------------------------------
运行: V3: C 语言循环重排版本 (i-k-j)
------------------------------------------------
矩阵大小: N = 1024
浮点运算次数: 2 * 1024^3 = 2147483648

========== V3: C 语言循环重排版本 (i-k-j) 结果 ==========
矩阵大小: 1024 x 1024
运行时间: 4.847139 秒
GFLOPS: 0.4430
================================================
```

图 2: V1-V3 运行结果

```
------------------------------------------------
运行: V4: C 语言编译优化版本 (-O3)
------------------------------------------------
矩阵大小: N = 1024
浮点运算次数: 2 * 1024^3 = 2147483648

========== V4: C 语言编译优化版本 (-O3) 结果 ==========
矩阵大小: 1024 x 1024
运行时间: 0.485981 秒
GFLOPS: 4.4189
================================================

------------------------------------------------
运行: V5: C 语言循环展开版本
------------------------------------------------
矩阵大小: N = 1024
浮点运算次数: 2 * 1024^3 = 2147483648

========== V5: C 语言循环展开版本 结果 ==========
矩阵大小: 1024 x 1024
运行时间: 0.340074 秒
GFLOPS: 6.3148
================================================

------------------------------------------------
运行: V6: Intel MKL 库版本
------------------------------------------------
矩阵大小: N = 1024
浮点运算次数: 2 * 1024^3 = 2147483648

========== V6: Intel MKL 库版本 结果 ==========
矩阵大小: 1024 x 1024
运行时间: 0.101739 秒
GFLOPS: 21.1078
```

图 3: V4-V6 运行结果

5　实验分析

5.1　循环顺序对性能的影响

i-j-k 与 i-k-j 循环顺序的性能差异分析：

- **i-j-k 顺序**：内层循环访问 `B[k][j]`，由于 C 语言数组按行存储，这是列优先访问模式，导致 Cache 命中率低。
- **i-k-j 顺序**：内层循环访问 `C[i][j]` 和 `B[k][j]`，都是行优先访问，Cache 命中率高。
- **性能提升**：版本 3 相比版本 2 的加速比为 **1.50 倍**，运行时间从 7.29 秒降至 4.85 秒。

5.2 编译优化的影响

`-O3` 编译选项带来的性能提升：

- 编译器自动进行循环展开、向量化、函数内联等优化
- 版本 4 相比版本 3 的加速比为 **9.98 倍**，运行时间从 4.85 秒降至 0.49 秒
- 这是所有优化中效果最显著的一步，说明编译器优化对性能至关重要

5.3 循环展开的影响

手动循环展开的优化效果：

- 减少循环控制开销（比较和跳转指令）
- 增加指令级并行性
- 版本 5 相比版本 4 的加速比为 **1.43 倍**，运行时间从 0.49 秒降至 0.34 秒
- 在编译优化基础上，手动循环展开仍能带来额外提升

5.4 MKL 库的性能优势

Intel MKL 库的性能分析：

- MKL 是 Intel 针对其 CPU 高度优化的数学库，包含多线程并行、SIMD 向量化、Cache 优化等技术
- 版本 6 相比版本 1 的绝对加速比为 **7.17 倍**，运行时间从 0.73 秒降至 0.10 秒
- 峰值性能百分比达到 **1.59%**，GFLOPS 达到 21.11
- 注意：由于测试使用的是 AMD CPU，MKL 的优化效果可能未完全发挥

5.5 Python NumPy 的性能表现

- Python (NumPy) 版本的 GFLOPS 为 2.94，远高于 C 语言基础版本 (0.29)
- 这是因为 NumPy 底层调用了优化的 BLAS 库，已经包含了向量化等优化
- 相比之下，未优化的 C 语言版本反而最慢

5.6 性能优化总结

1. **算法层面**：循环重排（i-k-j）可提升 Cache 命中率，加速比 1.50 倍
2. **编译层面**：`-O3` 优化效果最显著，加速比 9.98 倍
3. **代码层面**：循环展开在编译优化基础上仍有提升，加速比 1.43 倍
4. **库层面**：MKL 库性能最优，绝对加速比 7.17 倍（相对于 Python）

6 实验总结

6.1 注意事项

- 运行测试前需确保已安装 Intel MKL 库并设置环境变量
- 矩阵大小应足够大（≥ 512）以减少计时误差
- 多次测试取平均值可提高结果可靠性

6.2 心得体会

通过本次实验，我初步理解了以下内容：

- 矩阵乘法的多种实现方式及其性能差异
- Cache 命中率对程序性能的重要影响
- 专业数学库（如 MKL）的强大性能优势
- 性能评估指标（GFLOPS、加速比等）的计算方法

6.3 参考资料

1. Intel MKL 文档：`https://www.intel.com/content/www/us/en/docs/onemkl/tutorial-c/2021-4/multiplying-matrices-using-dgemm.html`
2. 矩阵乘法优化：`https://jackwish.net/2019/gemm-optimization.html`